

Week 11: Seeing with Networks: Convolutional Neural Networks and Computer Vision

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-11/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-11/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-11/lab.html>

Colab notebook with Python step 11 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-11/NB11_cnn_vision.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Cameras are cheap sensors, and much engineering knowledge is visual: cracks in concrete, defects on steel strip, grains on a conveyor, fibres in a fabric, cells under a microscope. Convolutional neural networks learn visual features directly from pixels by sharing small filters across the whole image [1, 2]. Since AlexNet won the ImageNet challenge, deep convolutional networks and their successors have transformed computer vision [3, 4, 5]. This week explains convolution, pooling and modern architectures, trains a crack detector on generated concrete images, compares networks on a clothing benchmark, reuses a pretrained network through transfer learning [6], and asks the network where it looks with Grad-CAM [7].

Estimated study time: 10 to 12 hours.

Learning outcomes

By the end of the week, students are expected to compute a convolution and a pooling step by hand, to calculate output sizes and parameter counts of convolutional layers, to build and train a small convolutional network in PyTorch, to explain why weight sharing makes convolutional networks efficient, to apply transfer learning and data augmentation to small engineering datasets, and to inspect a trained network with Grad-CAM and judge whether it relies on the right evidence.

Python in this week

These notes close with Python step 11: Images, batches and network classes. It covers images as arrays, cropping and flipping, stacking images into batches, the channel order of PyTorch, iterating with a

DataLoader, and networks written as classes, applied to crack images [8]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Images as engineering data

A digital image is an array of numbers. A greyscale image of 64 by 64 pixels holds 4096 intensities, and a colour image stores three such arrays, one per channel. Engineering images come from inspection cameras on production lines, drones over bridges and power lines, microscopes, satellites, thermal cameras and X-ray systems. Treating the pixels as 4096 unrelated inputs to a multilayer network ignores two facts: Neighbouring pixels belong together, and the same pattern, such as an edge or a crack, can appear anywhere in the image. Convolutional networks build both facts into their structure.

Formulas: Images as arrays and the cost of full connections

A greyscale image of H rows and W columns is an array $x[i, j]$, where i counts the rows from the top and j the columns from the left. A colour image has $C = 3$ such arrays, one per channel, and so holds $H \times W \times C$ numbers. Cameras usually store integers from 0 to 255, which are divided by 255 to give intensities between 0 and 1.

A fully connected layer that maps all $n_{\text{in}} = H W C$ inputs to n units needs a weight for every pair of input and unit:

$$P_{\text{dense}} = (H W C + 1) n$$

A convolutional layer with C_{out} filters of $K \times K$ weights, each looking at all C_{in} input channels, needs

$$P_{\text{conv}} = (K^2 C_{\text{in}} + 1) C_{\text{out}}$$

parameters, whatever the size of the image, because every filter is used at every position [1, 2].

Worked example: Pixels, parameters and weight sharing

Step 1. Counting the numbers. A greyscale image of 64 by 64 pixels holds $64 \times 64 = 4096$ intensities, and a colour image of the same size $64 \times 64 \times 3 = 12,288$ numbers. A stored pixel value of 178 becomes the intensity $178/255 = 0.698039$.

Step 2. A dense layer. A fully connected layer from the 4096 intensities to only 8 units has

$$P_{\text{dense}} = (4096 + 1) \times 8 = 32,768 + 8 = 32,776$$

parameters.

Step 3. A convolutional layer. Eight filters of 3 by 3 weights on the single channel have

$$P_{\text{conv}} = (3^2 \times 1 + 1) \times 8 = 10 \times 8 = 80$$

parameters. With a border of one pixel, each filter produces a feature map of 64 by 64 values, so the layer computes $8 \times 4096 = 32,768$ outputs. A fully connected layer with the same number of outputs would need

$$(4096 + 1) \times 32,768 = 134,250,496$$

parameters, about 1.7 million times as many. Weight sharing is what makes networks for images trainable on ordinary data sets.

Convolution, feature maps and pooling

A convolutional layer slides a small filter, for example 3 by 3 weights, over the image and computes at every position the weighted sum of the pixels under it. The result is a feature map that is large where the image locally resembles the filter. A vertical-edge filter responds to vertical edges wherever they are, because the same weights are used at every position. This weight sharing drastically reduces the number of parameters compared with a fully connected layer and makes the detector equivariant to shifts [1, 2]. A layer learns many filters at once, each producing its own feature map, or channel.

Three settings shape a convolution. The kernel size sets the local window, the stride sets the step between positions, and padding adds a border so that the output can keep the input size. For an input of width W , kernel K , padding P and stride S , the output width is $(W - K + 2P) / S + 1$. Pooling layers then summarise small neighbourhoods, usually by their maximum, which halves the resolution and makes the representation more tolerant to small shifts. Stacking convolution, activation and pooling lets later layers see larger parts of the image: Early layers detect edges and textures, later layers combine them into parts and objects [9].

Animation: A filter slides over an image. The highlighted 3 by 3 window moves across a small concrete image with a crack. Each output value is the sum of the pixels in the window multiplied by the filter weights. Choose different filters and compare the feature maps. Run it on the [lecture page](#).

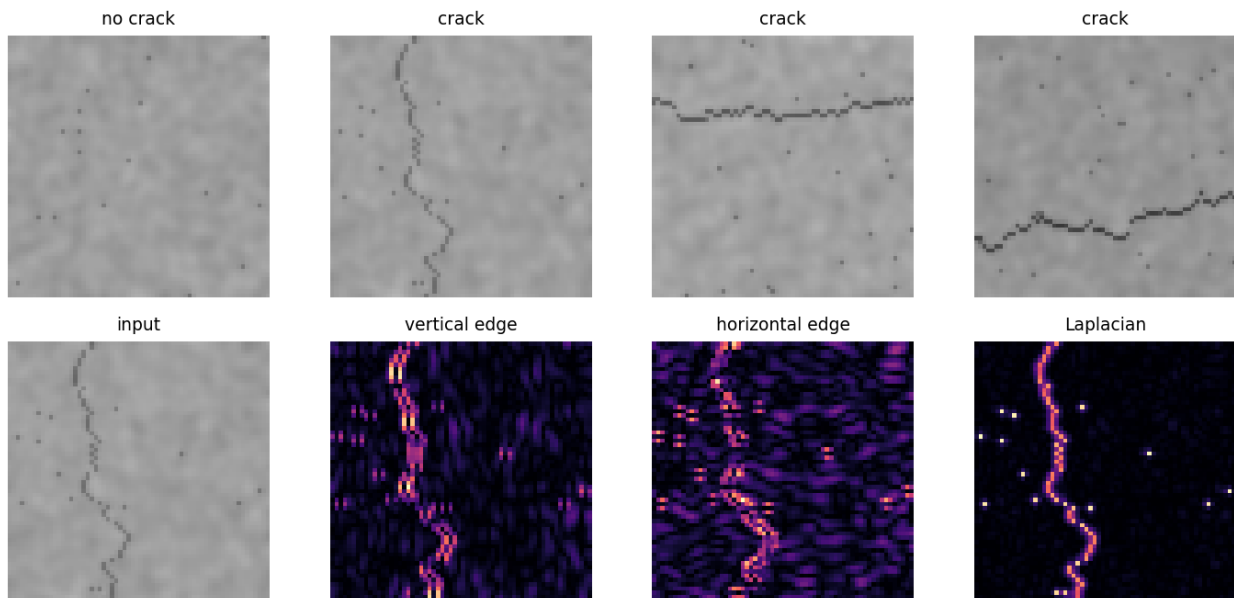


Figure 11.1. Generated concrete surfaces with and without cracks (top) and their responses to a vertical-edge filter, a horizontal-edge filter and a Laplacian filter (bottom).

Check your understanding. An input of 64 by 64 pixels passes a 5 by 5 convolution without padding and with stride 1, followed by 2 by 2 max pooling. What is the output size?

- A. 64 by 64
- B. 60 by 60
- C. 30 by 30
- D. 32 by 32

Answer: C. The convolution gives $(64 - 5 + 0) / 1 + 1 = 60$, and pooling halves it to 30.

Formulas: Convolution, output size, pooling and the receptive field

A convolutional layer slides a filter of $K \times K$ weights $w[a, c]$ over the input X and adds a bias b [2]. Deep learning libraries such as PyTorch compute the sum without flipping the filter, which mathematicians call a cross-correlation [8]:

$$Y[i, j] = b + \sum_{a=0}^{K-1} \sum_{c=0}^{K-1} w[a, c] X[iS+a, jS+c]$$

With the stride S , the window moves S pixels between neighbouring outputs. With C_{in} input channels, the filter has $K \times K$ weights for every channel, and the sum also runs over the channels. Padding adds P rows and columns of zeros around the input, and the output width then is

$$W_{out} = \frac{W - K + 2P}{S} + 1$$

rounded down when the division is not exact; the height follows from the same formula. An activation, usually ReLU, follows the convolution. Max pooling over windows of 2×2 with stride 2 keeps the largest value of each window and halves the width and the height, and average pooling keeps the mean.

The receptive field r of a unit is the size of the input window that can influence it. Starting from $r_0 = 1$ and a step of $s_0 = 1$ input pixel between neighbouring units, a layer with the window K_l and the stride S_l gives

$$r_l = r_{l-1} + (K_l - 1)s_{l-1}, s_l = s_{l-1}S_l$$

so that deeper layers see larger parts of the image [9].

Worked example: A convolutional block by hand

A 4 by 4 patch of a concrete photograph contains a dark vertical crack in its third column, with intensities

Row	Column 0	Column 1	Column 2	Column 3
0	0.9	0.8	0.2	0.7
1	0.8	0.8	0.1	0.8
2	0.9	0.7	0.1	0.9
3	0.8	0.9	0.3	0.6

The filter has the weights $1, -2, 1$ in each of its three rows and the bias $b = -0.5$. It compares the centre column of its window with the two neighbouring columns and responds to dark vertical lines. Because every row of the filter is the same, the sum of the nine products equals the left column sum of the window, minus twice the centre column sum, plus the right column sum.

Step 1. Output size. Without padding and with stride 1, the feature map has the width $(4 - 3 + 0)/1 + 1 = 2$, so it has 2 by 2 values.

Step 2. The column sums. The window of output (i, j) covers rows i to $i+2$ and columns j to $j+2$:

Output	Left column	Centre column	Right column
(0, 0)	$0.9+0.8+0.9=2.6$	$0.8+0.8+0.7=2.3$	$0.2+0.1+0.1=0.4$
(0, 1)	$0.8+0.8+0.7=2.3$	$0.2+0.1+0.1=0.4$	$0.7+0.8+0.9=2.4$
(1, 0)	$0.8+0.9+0.8=2.5$	$0.8+0.7+0.9=2.4$	$0.1+0.1+0.3=0.5$
(1, 1)	$0.8+0.7+0.9=2.4$	$0.1+0.1+0.3=0.5$	$0.8+0.9+0.6=2.3$

Step 3. The four weighted sums. With the bias:

$$Y[0, 0] = 2.6 - 2 \times 2.3 + 0.4 - 0.5 = -2.1$$

$$Y[0, 1] = 2.3 - 2 \times 0.4 + 2.4 - 0.5 = 3.4$$

$$Y[1,0]=2.5-2\times 2.4+0.5-0.5=-2.3$$

$$Y[1,1]=2.4-2\times 0.5+2.3-0.5=3.2$$

Step 4. ReLU and pooling. ReLU sets the two negative values to zero and keeps 3.4 and 3.2. Max pooling of the 2 by 2 map gives $\max(0, 3.4, 0, 3.2)=3.4$, and average pooling would give $(0+3.4+0+3.2)/4=1.65$. The strong responses sit where the centre column of the window lies on the crack, at $j=1$. Part C of the [interactive lab](#) computes every product of such a block for any input and filter.

Worked example: Sizes, parameters and receptive field of the crack network

The crack classifier of the notebook stacks three convolutions, Conv 1 to Conv 3, with 8, 16 and 32 filters of 3 by 3 weights, stride 1 and padding 1. Max pooling over 2 by 2 windows with stride 2, Pool 1 and Pool 2, follows the first two. The head averages each of the 32 last feature maps into one number and maps the 32 numbers to the two classes with a linear layer.

Step 1. Sizes and parameters.

Layer	Width	Output	Parameters
Input		$64 \times 64 \times 1$	
Conv 1	$(64 - 3 + 2)/1 + 1 = 64$	$64 \times 64 \times 8$	$(9 \times 1 + 1) \times 8 = 80$
Pool 1	$(64 - 2)/2 + 1 = 32$	$32 \times 32 \times 8$	0
Conv 2	$(32 - 3 + 2)/1 + 1 = 32$	$32 \times 32 \times 16$	$(9 \times 8 + 1) \times 16 = 1168$
Pool 2	$(32 - 2)/2 + 1 = 16$	$16 \times 16 \times 16$	0
Conv 3	$(16 - 3 + 2)/1 + 1 = 16$	$16 \times 16 \times 32$	$(9 \times 16 + 1) \times 32 = 4640$
Head		32, then 2	$32 \times 2 + 2 = 66$

Step 2. The receptive field. With the rule $r_l = r_{l-1} + (K_l - 1)s_{l-1}$ and $s_l = s_{l-1}S_l$:

Layer	K_l, S_l	r_l	s_l
Input		1	1
Conv 1	3, 1	$1 + 2 \times 1 = 3$	1
Pool 1	2, 2	$3 + 1 \times 1 = 4$	$2 \times 1 = 2$
Conv 2	3, 1	$4 + 2 \times 2 = 8$	2
Pool 2	2, 2	$8 + 1 \times 2 = 10$	$2 \times 2 = 4$
Conv 3	3, 1	$10 + 2 \times 4 = 18$	4

Step 3. Reading the tables. The convolutions have $80 + 1168 + 4640 = 5888$ parameters, and with the 66 parameters of the last layer the network has 5954. Every unit of the last feature map of 16 by 16 values sees a window of 18 by 18 pixels, about a quarter of the image width; the average pooling then combines the evidence of all positions. Part B of the lab computes such tables for other designs.

Deep architectures

LeNet showed in the 1990s that convolutional networks trained by backpropagation read handwritten digits reliably [2]. AlexNet, a deeper network trained on graphics processors with ReLU activations and dropout, won the 2012 ImageNet challenge by a wide margin [3, 4]. VGG networks used only small 3 by 3 filters stacked deeply [10]. Residual networks added shortcut connections that let a layer learn a correction to its input, which made networks with more than a hundred layers trainable [5]. Vision transformers split an image into patches and process them with the attention mechanism of Week 13; with enough data they match or exceed convolutional networks [11].

Formulas: Stacked small filters, residual connections and patches

A stack of n convolutions of 3 by 3 with stride 1 has the receptive field $(2n+1) \times (2n+1)$, by the rule of the previous section. With C channels in and out, it needs $n \times 9C^2$ weights, while a single convolution with the same receptive field needs $(2n+1)^2 C^2$. For $n=3$, this is $27C^2$ against $49C^2$, and the stack adds two more nonlinearities, which is why VGG networks are built from 3 by 3 filters [10].

A residual block adds its input to the output of a few layers, so that the layers learn a correction $F(x)$ rather than a whole new representation [5]:

$$y = F(x) + x$$

By the chain rule, the derivative of the loss passes the block as

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(\frac{\partial F}{\partial x} + 1 \right)$$

The added 1 lets gradients reach early layers even when $\partial F / \partial x$ is small, which made networks with more than a hundred layers trainable.

A vision transformer cuts an image of $H \times W$ pixels with C channels into N square patches of $p \times p$ pixels, flattens each patch into a vector and projects it linearly [11]:

$$N = \frac{H W}{p^2}, \text{ values per patch} = p^2 C$$

Worked example: Small filters, shortcuts and patches

Step 1. Three small filters or one large filter. With 64 channels in and out, one 3 by 3 convolution has $3 \times 3 \times 64 \times 64 + 64 = 36,928$ parameters, and three of them have $3 \times 36,928 = 110,784$. Their receptive field grows as $1+2=3$, $3+2=5$ and $5+2=7$. A single 7 by 7 convolution with the same receptive field has $7 \times 7 \times 64 \times 64 + 64 = 200,768$ parameters, so the stack needs only $110,784/200,768 = 0.551801$ of them.

Step 2. A residual block. A block computes $F(x) = w_2 \text{ReLU}(w_1 x)$ with the small weights $w_1 = 0.1$ and $w_2 = 0.2$. For the input $x = 2$:

$$F(2) = 0.2 \times \text{ReLU}(0.1 \times 2) = 0.2 \times 0.2 = 0.04, y = 0.04 + 2 = 2.04$$

Since the ReLU is active, its slope is 1, and the derivatives are

$$\frac{dF}{dx} = w_2 \times 1 \times w_1 = 0.2 \times 0.1 = 0.02, \frac{dy}{dx} = 0.02 + 1 = 1.02$$

Step 3. Twenty blocks. The gradient that passes 20 such blocks is multiplied by the product of their derivatives. Without shortcuts it shrinks to $0.02^{20} = 1.048576 \times 10^{-34}$ and vanishes; with shortcuts it is $1.02^{20} = 1.485947$, so the early layers still learn.

Step 4. Patches. An image of 224 by 224 pixels in three colours, cut into patches of 16 by 16 pixels, gives

$$N = \frac{224 \times 224}{16^2} = \frac{50,176}{256} = 196, 16^2 \times 3 = 768$$

so the transformer processes 196 vectors of 768 numbers, plus one extra vector whose output serves for the classification. Week 13 explains how attention combines them.

Training with little data

Engineering datasets are small compared with ImageNet: a few hundred labelled crack photographs, a few thousand defect images. Two techniques make deep networks usable nonetheless. Data augmentation creates plausible variants of the training images by flipping, rotating, cropping or changing brightness, which teaches invariances that the task requires. Transfer learning starts from a network pretrained on a large dataset and retrain only its last layers or fine-tunes all of them with a small learning rate; Yosinski and colleagues showed that early layers learn general features that transfer well between tasks [6]. Cha and colleagues trained a convolutional network on patches of concrete photographs to detect cracks under varied lighting [12], and similar studies inspect steel strip surfaces [13], crops and weeds [14] and traffic signs [15].

Check your understanding. A team has 400 labelled images of weld defects. Which approach is most promising?

- A. Train a very deep network from random initial weights without augmentation
- B. Fine-tune a network pretrained on a large image dataset, with data augmentation
- C. Use a multilayer perceptron on raw pixels
- D. Remove the test set to have more training data

Answer: B. Pretrained features and augmentation compensate for the small dataset [6].

Formulas: Augmentation, transfer learning and the softmax head

Augmentations are operations on the pixel grid that keep the label. For a square image of $W \times W$ pixels with rows and columns counted from 0, a mirror image and a rotation by 90 degrees anticlockwise take their values from

$$x_{\text{mirror}}[i, j] = x[i, W - 1 - j], x_{\text{rot}}[i, j] = x[j, W - 1 - i]$$

The four rotations by multiples of 90 degrees, with and without mirroring, give eight versions of every image. A change of contrast a and brightness b maps every intensity to $\text{clip}(ax + b)$, where clip keeps the result between 0 and 1.

Transfer learning keeps the layers of a network pretrained on a large data set, which compute a feature vector $\varphi(x)$ of length d , and adds a new final layer for the K classes of the task [6]. The new layer computes a score per class and the softmax function turns the scores into probabilities:

$$s_k = w_k \cdot \varphi(x) + b_k, \hat{p}_k = \frac{e^{s_k}}{\sum_{m=1}^K e^{s_m}}$$

Training minimises the cross-entropy $L = -\ln \hat{p}_y$ of the true class y , whose gradients are

$$\frac{\partial L}{\partial s_k} = \hat{p}_k - [k = y], \frac{\partial L}{\partial w_{kj}} = (\hat{p}_k - [k = y]) \varphi_j(x)$$

where $[k = y]$ is 1 for the true class and 0 otherwise. The new layer has $(d + 1)K$ parameters. When the pretrained layers are frozen, only these are trained, which suits data sets of a few hundred images.

Worked example: Eight versions of one image

Step 1. Mirror and rotation. A 3 by 3 image with a diagonal crack, $W = 3$, is mirrored and rotated by the formulas above:

Row	Original	Mirrored	Rotated by 90 degrees
0	0.1, 0.7, 0.8	0.8, 0.7, 0.1	0.8, 0.8, 0.3

Row	Original	Mirrored	Rotated by 90 degrees
1	0.7, 0.2, 0.8	0.8, 0.2, 0.7	0.7, 0.2, 0.8
2	0.8, 0.8, 0.3	0.3, 0.8, 0.8	0.1, 0.7, 0.8

For example, the mirrored value in row 0 and column 0 is $x[0, 2] = 0.8$, and the rotated value in row 2 and column 0 is $x[0, 2 - 2] = x[0, 0] = 0.1$. The crack runs from the top left to the bottom right in the original and from the top right to the bottom left in both new images, which are still valid examples of a crack.

Step 2. Contrast and brightness. With $a = 1.5$ and $b = -0.1$, the crack pixels become $1.5 \times 0.1 - 0.1 = 0.05$, $1.5 \times 0.2 - 0.1 = 0.2$ and $1.5 \times 0.3 - 0.1 = 0.35$, and the background pixels $1.5 \times 0.7 - 0.1 = 0.95$ and $1.5 \times 0.8 - 0.1 = 1.1$, which clip sets to 1. The notebook draws a random mirror and a random number of quarter turns for every batch, so that each of the eight versions appears with the probability $\frac{1}{2} \times \frac{1}{4} = \frac{1}{8}$.

Worked example: A new head on a frozen network

Step 1. Parameters. ResNet-18 has 11,689,512 parameters, of which its final layer for the 1000 ImageNet classes has $512 \times 1000 + 1000 = 513,000$ [4, 5]. The notebook replaces this layer by one for two classes, with $512 \times 2 + 2 = 1026$ parameters, so the network has $11,689,512 - 513,000 + 1026 = 11,177,538$ parameters. With the other layers frozen, only 1026 of them, 0.0092%, are trained on the 600 crack images.

Step 2. Softmax. For one image, the new layer gives the scores $s_0 = 0.4$ for no crack and $s_1 = 1.9$ for crack:

$$e^{0.4} = 1.491825, e^{1.9} = 6.685894, 1.491825 + 6.685894 = 8.177719$$

$$\hat{p}_0 = \frac{1.491825}{8.177719} = 0.182426, \hat{p}_1 = \frac{6.685894}{8.177719} = 0.817574$$

Step 3. Loss and gradients. The image shows a crack, $y = 1$, so $L = -\ln 0.817574 = 0.201414$. The gradients of the scores are $\hat{p}_0 - 0 = 0.182426$ and $\hat{p}_1 - 1 = -0.182426$: Training raises the crack score and lowers the other. A feature with the value $\phi_j = 1.3$ gives its weight to the crack score the gradient $-0.182426 \times 1.3 = -0.237154$. Part D of the [interactive lab](#) computes and applies these gradients for a small head.

Beyond classification

Classification assigns one label to an image. Object detection finds and labels every object with a bounding box; YOLO treats detection as a single regression problem and runs in real time [16]. Semantic segmentation labels every pixel; U-Net combines a contracting path with an expanding path and skip connections and became a standard for segmentation with few training images [17]. For a bridge inspector, a classifier says “crack present”, a detector draws a box around each crack, and a segmentation network outlines the crack so that its length and width can be measured.

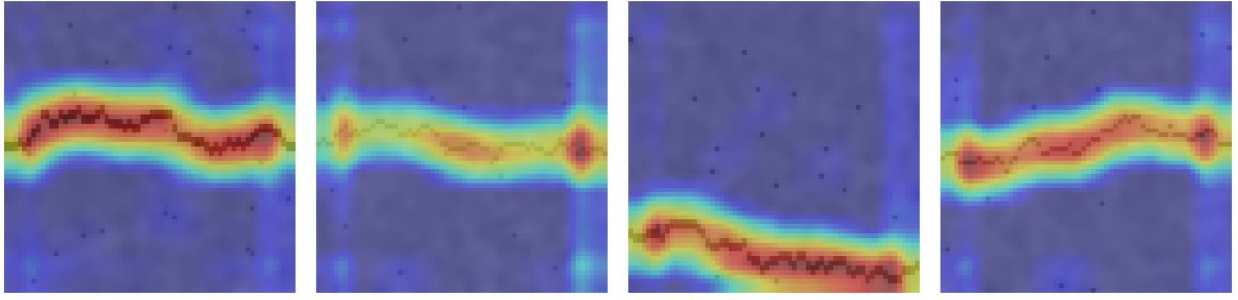


Figure 11.2. Grad-CAM heatmaps of the crack classifier trained in the notebook. The network attends to the crack pixels rather than to the background texture [7].

Formulas: Boxes, overlap, suppression and masks

A detector describes an object by a box with the corners (x_1, y_1) at the top left and (x_2, y_2) at the bottom right, in pixels. The overlap of a predicted box A with a true box B is measured by the intersection over union:

$$w_{\cap} = \max(0, \min(x_2^A, x_2^B) - \max(x_1^A, x_1^B))$$

$$h_{\cap} = \max(0, \min(y_2^A, y_2^B) - \max(y_1^A, y_1^B))$$

$$\text{IoU} = \frac{w_{\cap} h_{\cap}}{\text{area}(A) + \text{area}(B) - w_{\cap} h_{\cap}}$$

where the denominator is the area of the union. A detection usually counts as correct when its IoU with a true box reaches a threshold such as 0.5. Detectors propose many overlapping boxes, and non-maximum suppression cleans them up: It keeps the box with the highest confidence, removes every other box whose IoU with it exceeds a threshold, and repeats with the remaining boxes. YOLO divides the image into $S \times S$ cells, and each cell predicts B boxes, each with four coordinates and a confidence, together with C class probabilities, so that one pass of the network outputs $S \times S \times (5B + C)$ numbers [16].

A segmentation network such as U-Net labels every pixel [17]. Comparing a predicted mask with the true mask pixel by pixel gives the true positives TP, the false positives FP and the false negatives FN, and two overlap measures:

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}, \text{Dice} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}} = \frac{2 \text{IoU}}{1 + \text{IoU}}$$

With a known size g of a pixel on the surface, a mask of n pixels covers the area $n g^2$, and the mean width of a crack is its area divided by its length.

Worked example: Overlap of boxes and suppression

Step 1. IoU with the true box. An inspector marked a crack with the box $B = (40, 10, 70, 90)$. The detector proposes $A = (45, 20, 80, 95)$ with the confidence 0.9:

$$w_{\cap} = \min(70, 80) - \max(40, 45) = 70 - 45 = 25$$

$$h_{\cap} = \min(90, 95) - \max(10, 20) = 90 - 20 = 70$$

The areas are $25 \times 70 = 1750$ for the intersection, $30 \times 80 = 2400$ for B and $35 \times 75 = 2625$ for A :

$$\text{IoU} = \frac{1750}{2400 + 2625 - 1750} = \frac{1750}{3275} = 0.534351$$

which exceeds 0.5, so the detection counts as correct.

Step 2. Suppression. The detector also proposes $A' = (48, 15, 82, 92)$ with the confidence 0.7. Its overlap with A has $w_{\cap} = 80 - 48 = 32$, $h_{\cap} = 92 - 20 = 72$ and the area 2304, and A' has the area $34 \times 77 = 2618$:

$$\text{IoU}(A, A') = \frac{2304}{2625 + 2618 - 2304} = \frac{2304}{2939} = 0.78394$$

With a suppression threshold of 0.5, A is kept as the more confident box and A' is removed, so the crack is reported once. The removal also matters for the score: A' alone would have the IoU 0.489905 with the true box and would count as a miss.

Step 3. Size of a YOLO output. With $S = 7$, $B = 2$ and the $C = 20$ classes of the original paper, the network outputs $7 \times 7 \times (5 \times 2 + 20) = 49 \times 30 = 1470$ numbers per image.

Worked example: Measuring a crack from a mask

A segmentation network labels a patch of 6 by 6 pixels, each 0.5 mm wide on the concrete surface. Comparing its mask with the true mask gives, row by row:

Row	1	2	3	4	5	6
1			TP			
2			TP	FP		
3			FN	TP		
4				TP		
5				TP	FP	
6				TP		

Step 1. Counts. The table has TP = 6, FP = 2 and FN = 1, and the remaining $36 - 6 - 2 - 1 = 27$ pixels are true negatives.

Step 2. Overlap measures.

$$\text{IoU} = \frac{6}{6+2+1} = \frac{6}{9} = 0.666667, \text{Dice} = \frac{2 \times 6}{2 \times 6 + 2 + 1} = \frac{12}{15} = 0.8$$

which agrees with $2 \times 0.666667 / (1 + 0.666667) = 0.8$. The pixel accuracy $(6+27)/36 = 0.916667$ looks excellent only because most pixels are background, which is why segmentation is judged by IoU or Dice.

Step 3. Width of the crack. Each pixel covers $0.5 \times 0.5 = 0.25 \text{ mm}^2$. The predicted crack has $6+2=8$ pixels and the area $8 \times 0.25 = 2 \text{ mm}^2$; the true crack has $6+1=7$ pixels and 1.75 mm^2 . Both run through all six rows, a length of $6 \times 0.5 = 3 \text{ mm}$, so the mean width is $2/3 = 0.666667 \text{ mm}$ according to the network and $1.75/3 = 0.583333 \text{ mm}$ in truth. Part F of the [interactive lab](#) computes boxes, suppression and masks for any input.

Seeing what the network sees

High accuracy does not prove that a network uses the right evidence. A defect classifier may learn the lighting of the station where defective parts were photographed, or the ruler that appears only in images of damaged specimens. Grad-CAM weighs the feature maps of the last convolutional layer by the gradient of the class score and produces a coarse heatmap of the regions that drove the decision [7]. Checking such maps on a sample of test images, and testing the network on images from new cameras

and sites, belongs to every engineering deployment. Vision models are also vulnerable to small, deliberate perturbations of the input that change their decisions, which matters wherever an attacker could manipulate the images [18].

Check your understanding. A crack classifier reaches 99 percent test accuracy, but Grad-CAM shows that it attends to a timestamp printed in the corner of the images. What should be concluded?

- A. The classifier is excellent and can be deployed
- B. The classifier may rely on a shortcut that happens to correlate with the label, so the data and the evaluation must be revised
- C. Grad-CAM is broken
- D. The images should be larger

Answer: B. The explanation reveals a spurious cue; accuracy on data that share the cue does not guarantee performance elsewhere.

Formulas: Grad-CAM and the sensitivity to small perturbations

Grad-CAM explains the score y^c of a class c with the K feature maps A^k of the last convolutional layer, each of Z positions [7]. The weight of a map is the mean gradient of the class score over the map, and the heatmap is the weighted sum of the maps, of which ReLU keeps the positive part:

$$\alpha_k = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A^k[i, j]}, M[i, j] = \text{ReLU} \left(\sum_{k=1}^K \alpha_k A^k[i, j] \right)$$

The map has the coarse resolution of the last feature maps. It is enlarged to the size of the image and divided by its maximum for display, as in the notebook.

Small changes of many pixels can add up. For a linear score $s = w \cdot x + b$ over n pixels, a change δ of at most ϵ per pixel changes the score by at most

$$|\Delta s| = |w \cdot \delta| \leq \epsilon \sum_{i=1}^n |w_i|$$

and the bound is reached by $\delta_i = -\epsilon \text{sgn}(w_i)$, which lowers the score as far as possible. The bound grows with the number of pixels, which is one reason why image classifiers can be fooled by changes too small to see [18].

Worked example: Grad-CAM on two feature maps

The last layer of a crack classifier has two feature maps of 3 by 3 values. A^1 responds to the crack in the centre column and A^2 to the texture on the left:

Row	A^1	$\partial y^c / \partial A^1$	A^2	$\partial y^c / \partial A^2$
0	0, 1.2, 0.8	0.2, 0.4, 0.3	0.9, 0.2, 0	-0.1, 0, -0.2
1	0.1, 1.5, 0.9	0.3, 0.5, 0.3	0.8, 0.1, 0	-0.1, -0.1, 0
2	0, 1.1, 0.7	0.2, 0.4, 0.1	1, 0.3, 0.1	-0.2, 0, -0.2

Step 1. Weights of the maps. The gradients of the first map add up to 2.7 and those of the second to -0.9, so

$$\alpha_1 = \frac{2.7}{9} = 0.3, \alpha_2 = \frac{-0.9}{9} = -0.1$$

Step 2. Weighted sum, ReLU and normalisation. The weighted sums, their positive parts and the division by the largest value, 0.44, give

Position	$\alpha_1 A^1 + \alpha_2 A^2$	ReLU	Divided by 0.44
(0, 0)	$0.3 \times 0 + (-0.1) \times 0.9 = 0 - 0.09 = -0.09$	0	0
(0, 1)	$0.3 \times 1.2 + (-0.1) \times 0.2 = 0.36 - 0.02 = 0.34$	0.34	0.772727
(0, 2)	$0.3 \times 0.8 + (-0.1) \times 0 = 0.24 + 0 = 0.24$	0.24	0.545455
(1, 0)	$0.3 \times 0.1 + (-0.1) \times 0.8 = 0.03 - 0.08 = -0.05$	0	0
(1, 1)	$0.3 \times 1.5 + (-0.1) \times 0.1 = 0.45 - 0.01 = 0.44$	0.44	1
(1, 2)	$0.3 \times 0.9 + (-0.1) \times 0 = 0.27 + 0 = 0.27$	0.27	0.613636
(2, 0)	$0.3 \times 0 + (-0.1) \times 1 = 0 - 0.1 = -0.1$	0	0
(2, 1)	$0.3 \times 1.1 + (-0.1) \times 0.3 = 0.33 - 0.03 = 0.3$	0.3	0.681818
(2, 2)	$0.3 \times 0.7 + (-0.1) \times 0.1 = 0.21 - 0.01 = 0.2$	0.2	0.454545

The heatmap lights up the centre column, the crack, and ignores the left column, whose map has a negative weight. If it instead lit up a timestamp or a ruler, the classifier would be right for the wrong reason. Part G of the [interactive lab](#) computes the heatmap for any maps and gradients.

Worked example: A perturbation too small to see

A linear crack detector scores images of 64 by 64 pixels, $n = 4096$, with weights of size $|w_i| = 0.01$, and reports a crack when $s > 0$. A crack image has the score $s = 0.3$. A change of every intensity by two grey levels, $\epsilon = 2/255 = 0.007843$, against the sign of its weight lowers the score by

$$\epsilon \sum_{i=1}^n |w_i| = 0.007843 \times 4096 \times 0.01 = 0.007843 \times 40.96 = 0.321249$$

to $0.3 - 0.321249 = -0.021249$, and the crack is no longer reported. No pixel has changed visibly, yet the decision has flipped; a change of $0.3/40.96 = 0.007324$ per pixel, less than two grey levels, already suffices. Part H of the lab shows such a perturbation on a small image.

Python step 11: Images, batches and network classes

Images are arrays

A greyscale image is a two-dimensional array of intensities, usually scaled to the range 0 to 1, with the shape (height, width). Colour images add a third axis for the red, green and blue channels. Everything from Weeks 4 and 9 applies: Slicing crops an image, reversing an axis mirrors it, and masks find the pixels that meet a condition. The practice image below is light grey with a dark vertical crack in column 4.

```
import numpy as np

img = np.full((8, 10), 0.7)           # light grey image, 8 rows and 10
columns
img[:, 4] = 0.1                       # a dark crack in column 4
print(img.shape, img.min(), img.max())
crop = img[2:6, 3:6]                  # rows 2 to 5, columns 3 to 5
print(crop)
print("dark pixels:", int((img < 0.3).sum()))
flipped = img[:, ::-1]                 # mirror from left to right
print("crack column after flipping:",
int(np.argmax(flipped.mean(axis=0))))
```

Output

```
(8, 10) 0.1 0.7
[[0.7 0.1 0.7]
 [0.7 0.1 0.7]
 [0.7 0.1 0.7]
 [0.7 0.1 0.7]]
dark pixels: 8
crack column after flipping: 5
```

The slice `:: -1` steps backwards through an axis. Flips and small rotations are the data augmentation of this week: They create new training images that are still valid examples of a crack.

From images to batches

Networks process several images at once. `np.stack` joins images of equal shape along a new first axis, which gives the shape (number of images, height, width). PyTorch convolution layers expect a channel axis in second place, (batch, channels, height, width), so greyscale images receive an axis of length one with `[:, None]`. Libraries that read colour image files usually return (height, width, channels), and `permute` reorders the axes of a tensor.

```
import torch

batch = np.stack([img, flipped, img * 0.9])           # three images
print(batch.shape)
xb = torch.tensor(batch[:, None], dtype=torch.float32)
print(xb.shape)
rgb = torch.zeros(64, 64, 3)                         # colour image as
stored in files
print(rgb.permute(2, 0, 1).shape)                     # channels first,
as PyTorch expects
```

Output

```
(3, 8, 10)
torch.Size([3, 1, 8, 10])
torch.Size([3, 64, 64])
```

Iterating in batches

A `DataLoader` serves a dataset in batches of a chosen size, optionally in shuffled order, and a `for` loop receives one batch of images and labels per pass. `enumerate` numbers the batches. The last batch is smaller when the batch size does not divide the number of examples.

```
from torch.utils.data import TensorDataset, DataLoader

images = torch.rand(10, 1, 8, 10)
labels = torch.tensor([0, 1] * 5)
loader = DataLoader(TensorDataset(images, labels), batch_size=4,
shuffle=False)
for i, (xb_i, yb_i) in enumerate(loader):
    print(i, tuple(xb_i.shape), yb_i.tolist())
```

Output

```
0 (4, 1, 8, 10) [0, 1, 0, 1]
1 (4, 1, 8, 10) [0, 1, 0, 1]
2 (2, 1, 8, 10) [0, 1]
```

The expression `[0, 1] * 5` repeats a list five times, the list behaviour that NumPy arrays replace with element-wise arithmetic.

A network is a class

Week 6 introduced classes. A PyTorch network is a class that inherits from `nn.Module`: It gains all the machinery of a network and adds its own layers in `__init__` and its computation in `forward`. The call `super().__init__()` first runs the initialisation of the parent class. Calling the object, as in `net(xb)`, runs `forward`.

```
from torch import nn
```

```
class TinyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 4, kernel_size=3, padding=1)
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(4, 2)

    def forward(self, x):
        x = torch.relu(self.conv(x))
        return self.fc(self.pool(x).flatten(1))
```

```
net = TinyCNN()
print(net(xb).shape)
print(sum(p.numel() for p in net.parameters()), "parameters")
```

Output

```
torch.Size([3, 2])
50 parameters
```

The convolution has 4 filters of 3 by 3 weights plus 4 biases, 40 parameters, and the linear layer has 4 times 2 weights plus 2 biases, 10 parameters.

Check your understanding. Which shape does PyTorch expect for a batch of 16 colour images of 32 by 32 pixels?

- A. (16, 32, 32, 3)
- B. (16, 3, 32, 32)
- C. (3, 16, 32, 32)
- D. (32, 32, 3, 16)

Answer: B. Convolution layers expect batch, channels, height and width, in this order.

Discipline challenges

Images are part of almost every discipline. Choose a row and adapt the notebook's crack classifier or its transfer-learning section to the images named there.

Department	Challenge
Civil Engineering	Crack detection on concrete or pavement photographs, following Cha and colleagues [12].
Mechanical Engineering and Textile Engineering	Surface defect classes on hot-rolled steel strip or fabric [13].
Food Engineering and Biology	Grain, fruit or leaf disease images with transfer learning [14].
Automotive Engineering and Computer	Traffic sign recognition with the German

Department	Challenge
Engineering	benchmark [15].
Geological Engineering and Earth Sciences Engineering	Rock thin-section or outcrop images classified by lithology.
Environmental Engineering	Satellite or drone image tiles classified by land cover or pollution events.
Mining Engineering	Fragmentation images after blasting, classified by size class.
Electrical and Electronics Engineering	Printed circuit board defect images or thermal images of electrical panels.
Physics and Chemistry	Microscopy images of materials or crystals classified by phase or morphology.
Mathematics and Statistics	Show that convolution is linear and shift-equivariant and that max pooling is neither linear nor equivariant.

Weekly task

Train and evaluate a convolutional network for an image task related to your department. Use the generated crack images, the clothing or shape benchmark of the notebook, or a public engineering image dataset whose licence allows teaching use. Report the architecture, parameter count, augmentation, learning curves and test metrics, compare training from scratch with transfer learning, and show Grad-CAM maps for correct and wrong predictions. Discuss in about 500 words what would be needed for use on real site images, citing at least three works from this week's references [5, 6, 7].

Research and report assignment (optional)

Computer vision for inspection. Review deep learning for visual inspection in one domain, such as concrete and pavement cracks, steel surface defects, fabric defects, agricultural products or electronics. Compare datasets, architectures, the use of transfer learning and augmentation, and how the studies tested generalisation to new sites or cameras [12, 13, 14].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] LeCun, Y., Boser, B., Denker, J. S., et al. (1989). Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4), 541-551. <https://doi.org/10.1162/neco.1989.1.4.541>
- [2] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324. <https://doi.org/10.1109/5.726791>
- [3] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6), 84-90. <https://doi.org/10.1145/3065386>
- [4] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 248-255). <https://doi.org/10.1109/CVPR.2009.5206848>

- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770-778).
<https://doi.org/10.1109/CVPR.2016.90>
- [6] Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks?. In *Advances in Neural Information Processing Systems 27* (pp. 3320-3328).
<https://arxiv.org/abs/1411.1792>
- [7] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *2017 IEEE International Conference on Computer Vision (ICCV)* (pp. 618-626). <https://doi.org/10.1109/ICCV.2017.74>
- [8] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024-8035).
<https://arxiv.org/abs/1912.01703>
- [9] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
<https://doi.org/10.1038/nature14539>
- [10] Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *3rd International Conference on Learning Representations (ICLR 2015)*.
<https://arxiv.org/abs/1409.1556>
- [11] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations (ICLR 2021)*.
<https://arxiv.org/abs/2010.11929>
- [12] Cha, Y.-J., Choi, W., & Büyüköztürk, O. (2017). Deep learning-based crack damage detection using convolutional neural networks. *Computer-Aided Civil and Infrastructure Engineering*, 32(5), 361-378.
<https://doi.org/10.1111/mice.12263>
- [13] Song, K., & Yan, Y. (2013). A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science*, 285, 858-864.
<https://doi.org/10.1016/j.apsusc.2013.09.002>
- [14] Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). Deep learning in agriculture: A survey. *Computers and Electronics in Agriculture*, 147, 70-90. <https://doi.org/10.1016/j.compag.2018.02.016>
- [15] Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32, 323-332.
<https://doi.org/10.1016/j.neunet.2012.02.016>
- [16] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779-788). <https://doi.org/10.1109/CVPR.2016.91>
- [17] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015), LNCS 9351* (pp. 234-241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28
- [18] Kose, U. (2019). Techniques for adversarial examples threatening the safety of artificial intelligence based systems. In *I. International Science and Innovation Congress (INSI Congress 2019)*. Pamukkale, Denizli, Türkiye. <https://arxiv.org/abs/1910.06907>